

UAV-based sustainable orchard management: deep learning for apple detection and yield estimation

Alexey Kuttyrev*, Dmitry Khort, Igor Smirnov, and Valeria Zubina

Federal Scientific Agroengineering Center VIM, 109428 Moscow, Russia

Abstract. This article presents a method for automated apple counting using high-resolution images obtained from unmanned aerial vehicles (UAVs). The YOLO11 architecture, specifically models from YOLO11n to YOLO11x, was employed for fruit detection. Key steps included creating orthophotos, segmenting data into tiles, training a convolutional neural network (CNN) with transfer learning and data augmentation, and merging results. Images were captured using a DJI Mavic 3 Multispectral drone with a 20 MP RGB camera. Data augmentation including flipping, hue adjustment, blurring, and Tile 8×8 transformation increased the dataset from 11 to 2,000 images with 51,797 objects (34,383 apples and 17,414 fallen apples). The YOLO11x model achieved the highest performance metrics: mAP@50 = 0.816, mAP@50-95 = 0.547, Precision = 0.852, and Recall = 0.766, demonstrating its effectiveness in complex, high-density orchards. The YOLO11n model, with lower computational demands, is suitable for resource-limited environments. The method maintains geospatial alignment and visualizes fruit distribution across the orchard. An experimentally determined correction coefficient will account for fruits hidden from the camera, enhancing the accuracy of yield estimation. A Tkinter interface displays detection results and summary data for each orchard section. Future work includes integrating multispectral data and 3D modeling to enhance precision. These findings highlight the potential of deep learning to automate orchard monitoring and yield assessment.

1 Introduction

Traditional methods for inspecting orchards to assess yields rely on visual surveys conducted by agronomists. The main drawback of this approach is its subjectivity: human factors lead to significant errors in yield forecasting [1]. This reduces the accuracy and reproducibility of results and makes the process labor-intensive and inefficient on a production scale. Various technical solutions have been proposed to address these limitations.

Ground sensors and stationary RGB cameras enable continuous monitoring of specific orchard areas but are limited in coverage and flexibility for data collection [2]. Hyperspectral and thermal imaging facilitate early detection of plant stress and diseases but require complex equipment and specialized data processing techniques [3,4]. Laser scanning (LiDAR)

* Corresponding author: alexeykuttyrev@gmail.com

provides highly accurate information about tree structures and biomass, but its use is constrained by high costs and complex data interpretation [5].

In recent years, unmanned aerial vehicles (UAVs) have become widely used in agriculture due to their ability to collect data quickly and accurately, advancing precision farming technologies. UAVs automate the monitoring of plant conditions, disease diagnostics, and yield estimation [6]. However, most existing approaches to remote plant monitoring using UAVs are optimized for field crops or forests, making them less suitable for intensive orchards with high planting density.

Monitoring fruit orchards, particularly apple orchards, imposes specific requirements on data collection and processing technologies. UAVs are especially valuable for apple orchards, where high accuracy is needed to assess the condition of individual trees and fruits for yield prediction [7]. Intensive orchards with high planting density and regular planting structures pose challenges for fruit detection due to light interactions with canopies, overlapping fruits and leaves, and complex plant textures. Additionally, lighting variations and shadows can significantly affect image quality, complicating object recognition [8].

Effective use of UAV data requires modern image processing and analysis methods. Various computer vision and machine learning algorithms, including segmentation and classification, have been applied for plant identification and condition assessment [9]. Deep learning methods, particularly convolutional neural networks (CNNs), excel in these tasks by automatically extracting features and adapting to varying imaging conditions [10]. YOLO (You Only Look Once) architectures stand out for their high speed and accuracy, which are critical for processing large datasets collected by UAVs [11-14]. Other models, such as Faster R-CNN, SSD (Single Shot MultiBox Detector) [14], and RetinaNet [13] have also demonstrated high accuracy for agricultural object detection using UAVs. Transfer learning methods [18] further enhance detection performance by pre-training models on large datasets.

CNN-based models and their architectures are widely used to detect and count fruits in orchards, including apples, citrus, and vineyards. These models enable accurate detection of small objects in high-resolution images, compensating for detection errors caused by scene complexity and spatial variations [11]. To improve model accuracy under varying lighting conditions and object orientations, preprocessing techniques such as perspective correction and distortion removal from UAV altitude changes are employed [8]. These approaches significantly improve detection in dense canopies where fruits are partially obscured by leaves [8]. Thus, developing adapted methods for high-resolution orchard images obtained from UAVs is a relevant and timely task.

This study aims to develop a method for automated apple counting in high-resolution UAV images using the YOLO11 architecture to improve accuracy and efficiency in industrial orchard analysis.

2 Materials and methods

A comprehensive method has been developed for automated apple counting on high-resolution images obtained using UAVs. The method consists of several stages, each aimed at recognizing apple fruits in the conditions of an industrial orchard.

At the first stage (data preprocessing), aerial photographs obtained from UAVs were used to create high-resolution orthophotomaps using Agisoft Metashape software. The DJI Mavic 3 Multispectral (Mavic 3M) was used for the survey; only the RGB camera with a CMOS sensor (4/3", 20 MP), a focal length of 24 mm, an aperture of f/2.8, a field of view of 84°, and a maximum resolution of 5280×3956 pixels was utilized. The images were captured at an altitude of 30–60 meters under natural lighting conditions (clear weather, daytime), enabling detailed aerial photographs with sufficient resolution for detecting small objects, suitable for subsequent photogrammetric processing and analysis.

The Agisoft Metashape software enabled photogrammetric processing of the images, including image alignment, dense point cloud generation, and orthomosaic creation. During the image alignment process, algorithms for matching key points and camera optimization were used to ensure accurate image alignment. As a result, an orthophotomap of the orchard was generated, providing a detailed representation of the study area (an industrial orchard with ripened apple fruits) (fig.1).

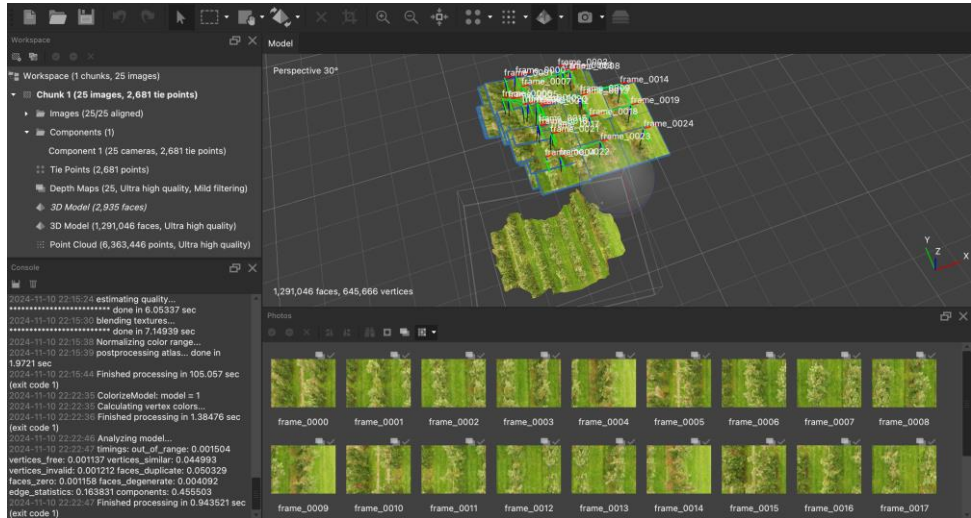


Fig. 1. Creating an orthophotomap of the orchard in Agisoft Metashape.

At the second stage, to optimize subsequent processing and reduce computational load, the high-resolution orthophotomap (GeoTIFF .tif format) was divided into fragments of fixed size. This process was carried out using a Python script with the PIL (Python Imaging Library) library. The division of the image into tiles is described by formula 1:

$$I_{ji} = I[x_i: x_{i+1}, y_i: y_{i+1}] \quad (1)$$

where I_{ji} — is an individual tile, x_i and y_i — are the coordinates defining the regions of the image division.

The parameters of the division included tile sizes of 200×200 pixels to maintain a balance between image detail and the volume of data for processing. To ensure correct positioning during subsequent stitching, each tile was assigned a unique name containing information about its position along the X and Y axes on the original orthophotomap image, which facilitates further processing and subsequent frame stitching.

At the third stage, a trained convolutional neural network model based on the YOLO11 architecture was applied to each of the obtained image fragments for apple fruit detection. The model was implemented using the PyTorch framework and trained with the application of transfer learning methods to improve training efficiency. Image annotation was performed using a bounding box tool. Two classes were designated as detection targets: apples on the tree, labeled as the «apple» class, and fallen apples, labeled as the «fallen apple» class. Annotation was performed manually using the Roboflow service (Bounding Boxes tool), which ensured high-quality data required for training the model (fig. 2).



Fig. 2. Dataset Annotation.

To expand the dataset, eliminate class imbalance, and increase the model's robustness to changes in lighting and object orientation, image augmentation was performed using the following methods: Tile 8x8 grid transformation, horizontal and vertical flipping (flip horizontal and flip vertical), hue adjustment with an intensity of up to 15%, and image blurring (Blur) with a coefficient of 1%. These methods allowed for diversifying the original data, increasing its volume, and improving the model's robustness to object variations in real-world conditions (fig.3).

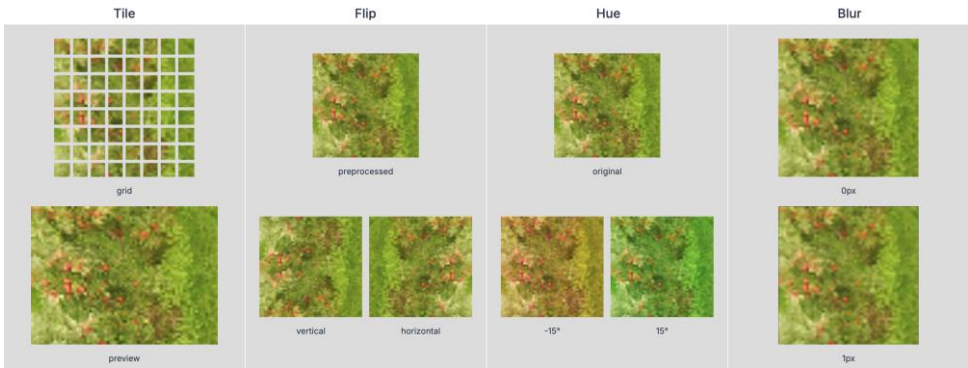


Fig. 3. Methods of Artificial Data Augmentation.

As a result of applying augmentation methods, the original dataset, consisting of 11 high-resolution images of 5280×3956 pixels, was expanded to 2000 images, significantly increasing the volume and diversity of the training data and improving the model's generalization ability. The total number of labeled objects for training amounted to 51,797, of which 34,383 belong to the «apple» class (apples on the tree) and 17,414 to the «fallen_apple» class (fallen apples). For training the model, pre-trained YOLO11 models were used, including YOLO11n (39.5% mAPval, 2.6M parameters, 6.5 GFLOPs), YOLO11s (47.0% mAPval, 9.4M, 21.5 GFLOPs), YOLO11m (51.5% mAPval, 20.1M, 68 GFLOPs), YOLO11l (53.4% mAPval, 25.3M, 86.9 GFLOPs), and YOLO11x (54.7% mAPval, 56.9M, 194.9 GFLOPs).

For optimizing the hyperparameters of the YOLO11 model, a batch size of 12, a learning rate of 0.1, and 1500 epochs were used. To prevent model overfitting, early stopping was applied when no improvement in metrics was observed on the validation dataset. To eliminate

duplicate or false detections, the Non-Maximum Suppression (NMS) algorithm was used, which excludes overlapping bounding boxes with low confidence, leaving only the most probable ones. The quality assessment of the models was carried out using metrics such as Precision, Recall, and mean Average Precision (mAP) at various IoU thresholds.

At the fourth stage, after detecting apples on individual tiles, the images were merged back into a single orthophotomap with the identified objects (apple fruits) highlighted by bounding boxes. This process was performed using a Python script with the PIL and OS libraries for image processing and file management. During stitching, the tile coordinates extracted from their filenames were used, ensuring accurate restoration of the geometry of the original image. Overlaps between tiles were taken into account to correctly align the detected objects. To prevent double counting errors, intersection elimination algorithms were applied. During stitching, the data on the number of detected fruits obtained in the previous stage were summed. The total number of apples N_{total} was calculated as the sum of apples on each tile according to formula 2:

$$N_{total} = \sum_{i=1}^n n_i \tag{2}$$

where n_i – is the number of detected fruits on each tile, pcs, and n – is the total number of tiles, pcs.

For training convolutional neural networks and data analysis, modern tools and libraries were used: CUDA 12.2, TensorFlow, TensorBoard, and Keras (version 2.15.0), PyArrow, JAX and JAXLib, NumPy, OpenCV, matplotlib, and tqdm. The model training was conducted on a computer with an Intel Core i9-10900X processor (3.70 GHz), two GeForce RTX 2080TI graphics cards (11 GB of video memory each), a GIGABYTE X299 UD4 Pro motherboard, an Intel PCI-E SSD (1 TB), and 128 GB of Kingston DDR4 RAM.Φ

3 Results and discussion

As a result of training the YOLO11n-x models, an evaluation of the Precision, Recall, and mean Average Precision (mAP) metrics was conducted at various IoU thresholds to analyze the accuracy and completeness of object classification. The mAP metric was calculated at standard IoU thresholds, including mAP@50 and mAP@50-95, providing a comprehensive assessment of the model's performance. On the mAP50-Iteration graph, which illustrates changes in average precision at a fixed IoU threshold of 50, it is observed that the YOLO11l and YOLO11x models achieve the highest values (around 0.8). This indicates their high ability to accurately detect objects in the images. The YOLO11n and YOLO11s models, which have simplified architectures, show lower average precision values (fig.4).

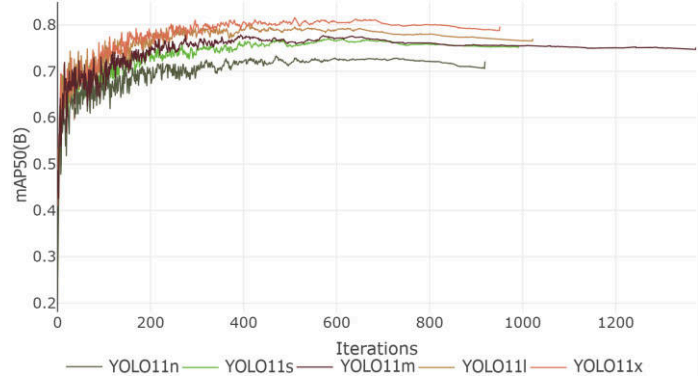


Fig. 4. mAP50-Iteration Graph.

The mAP50-95-Iteration graph, which includes mAP at various IoU thresholds from 50% to 95%, provides a more comprehensive assessment of detection quality. The best results are achieved by the YOLO11l and YOLO11x models, whose mAP values stabilize within the range of 0.5–0.6, indicating high object detection accuracy at various overlap levels (fig.5). The YOLO11n and YOLO11s models demonstrate lower mAP values but show stable metric growth during the initial stages of training, making them promising for use on devices with limited computational resources.

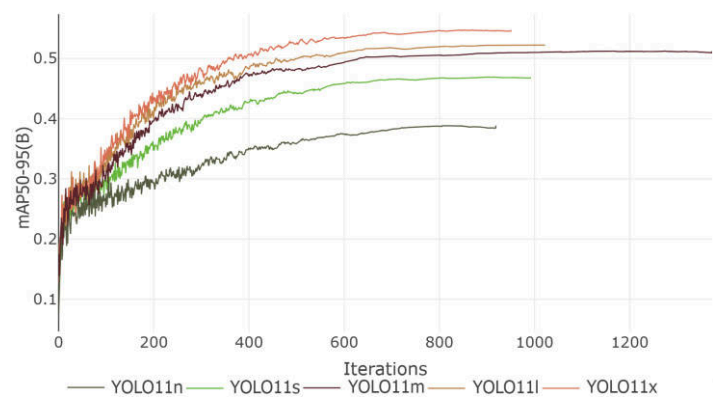


Fig. 5. mAP50-95-Iteration Graph.

In the Precision-Iteration graph, which reflects the proportion of correct detections among all predicted objects, the YOLO11x and YOLO11l models achieve values of approximately 0.8, highlighting their ability to minimize false positives (fig.6).

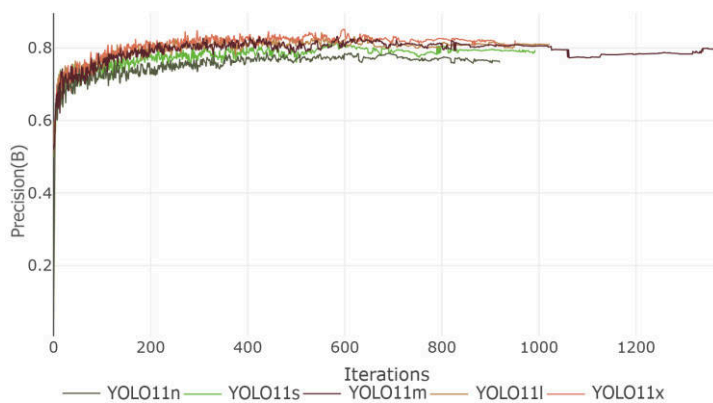


Fig. 6. Precision-Iteration Graph.

The Recall-Iteration graph shows the proportion of correct detections among all true objects, confirming the superiority of the YOLO11x and YOLO11l models, which achieve values of 0.7–0.8 (fig.7). This demonstrates the high capability of these models to detect even partially occluded or poorly visible objects. Models of lower complexity (YOLO11n, YOLO11s) exhibit lower values.

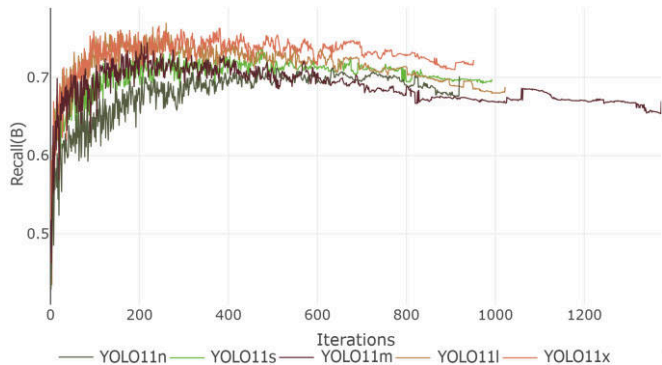


Fig. 7. Recall-Iteration Graph.

The key stabilization points in the training dynamics for all models are in the range of 600 to 800 iterations, indicating the achievement of stable parameter convergence and loss minimization. Within this range, the models demonstrate maximum efficiency in weight optimization, after which the rate of loss reduction significantly slows down. Further increasing the number of iterations leads to minimal changes in performance, indicating the completion of the main training phase. The YOLO11x and YOLO11l models show the best results in terms of losses for both localization and classification, making them preferable for tasks requiring high accuracy (fig.8,9). Simpler models, such as YOLO11n, exhibit slower loss reduction and less accurate results, but their use is justified in conditions with limited computational resources.

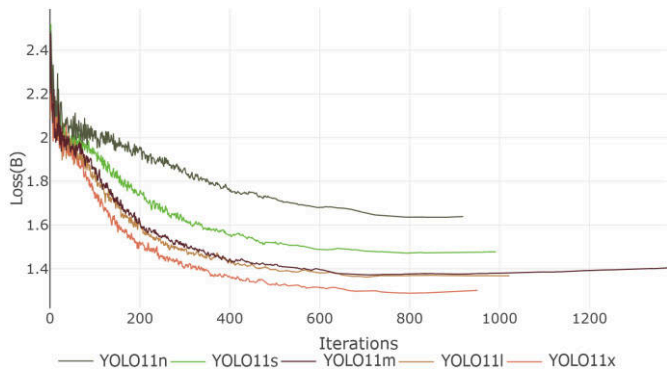


Fig. 8. Loss(B)-Iteration Graph.

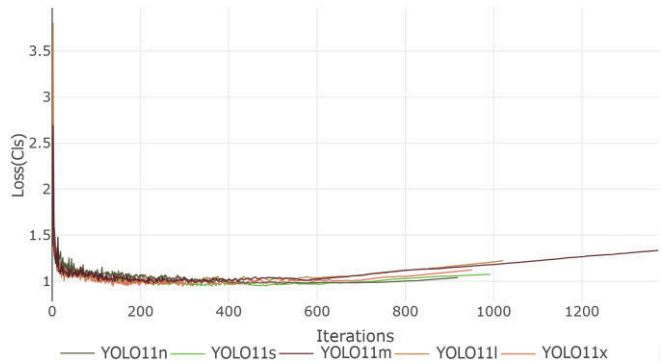


Fig. 9. Loss (Cls)-Iteration Graph.

Figure 10 shows examples of object detection on individual fragments (tiles) of the image, as well as the final orthophotomap of the orchard obtained by merging all processed tiles.

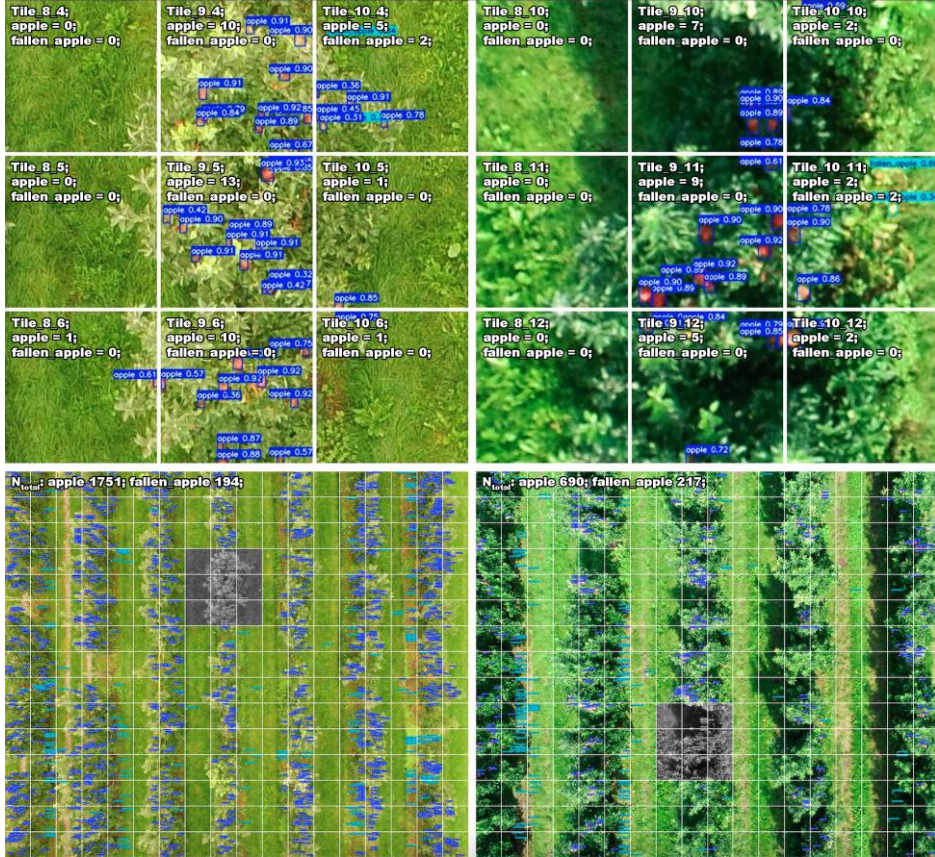


Fig. 10. Examples of apple fruit detection using the YOLO11x model and the final orthophotomap of the orchard.

For each tile, i.e., part of the high-resolution orthophotomap, data on the location and number of detected fruits were generated. Predictions for each tile included bounding box coordinates and the corresponding probabilities of fruit detection. The coordinates of the detected fruits are presented as a dataset, defined by formula 3:

$$\{(x_{ij}, y_{ij}) \mid j = 1, 2, \dots, N_i\} \quad (3)$$

where (x_{ij}, y_{ij}) – coordinates of the j -th apple fruit on the j -th tile.

This approach allows visualizing the detected objects across the entire area of the orchard, maintaining spatial reference and providing the user with data for further analysis of fruit density and distribution uniformity, as well as planning agronomic activities. The results of the metric calculations are presented in Table 1.

The YOLO11x model demonstrated the highest performance metrics, with mAP50 (0.81602), mAP50-95 (0.54721), Precision (0.85193), and Recall (0.76625), confirming its efficiency in tasks requiring high detection accuracy. YOLO11l offers an optimal balance between performance (mAP50 0.80041) and computational costs, making it a suitable choice for resource-constrained scenarios. YOLO11m exhibited balanced efficiency (mAP50 0.77836), making it applicable for medium-complexity tasks. YOLO11s and YOLO11n

models, although characterized by lower metric values, can be applied in environments with limited computational resources due to their simpler architecture and minimal resource requirements.

Table 1. Results of Metric Calculations.

Metric	YOLO11n	YOLO11s	YOLO11l	YOLO11m	YOLO11x
Mean Average Precision (mAP50)	0,73	0,77	0,80	0,77	0,81
Mean Average Precision (mAP50-95)	0,38	0,46	0,52	0,51	0,54
Precision	0,78	0,81	0,84	0,83	0,85
Recall	0,71	0,73	0,77	0,74	0,76
Classification Loss (val)	2,58	2,17	3,81	2,69	1,61
Bounding Box Loss (val)	2,48	2,52	2,41	2,47	2,43
Classification Loss (train)	2,39	1,79	1,69	1,81	1,48
Bounding Box Loss (train)	2,02	1,85	1,70	1,76	1,65

For user convenience, a graphical interface was developed using the Tkinter library, enabling users to view processing results and obtain summary data for each orchard section from text files containing aggregate information about the number of detected apple fruits (fig.11).

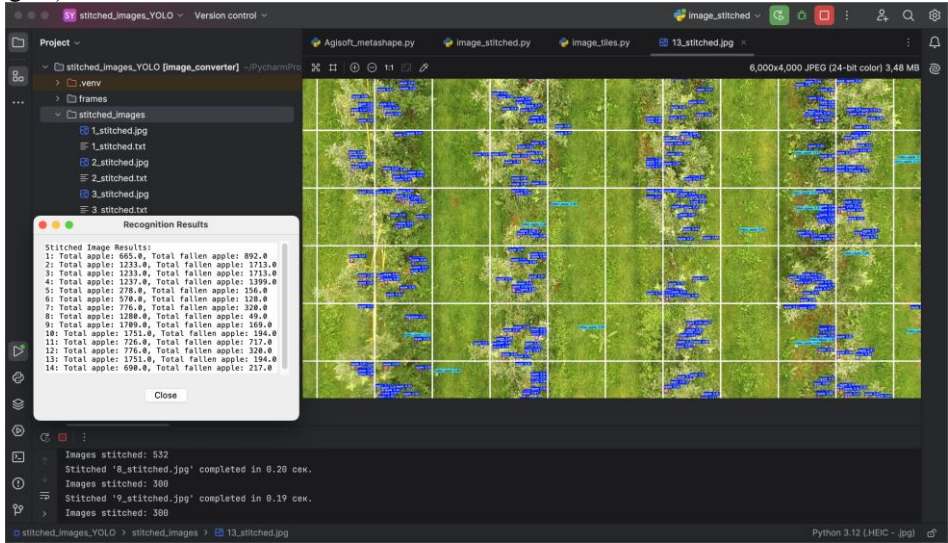


Fig. 11. Image processing results, summary data for each orchard section.

The interface allows visualizing detections on the combined image and exporting data in various formats for further analysis.

4 Conclusions

The developed method provides automated apple counting in industrial orchards with high planting density and challenging shooting conditions, using data obtained from UAVs. The application of the YOLO11 model at the stage of processing individual tiles allowed for reducing computational load while maintaining high detection accuracy due to detailed analysis of each image fragment. Post-processing, including the merging of detections, ensures the integrity of geospatial information, enabling the visualization of fruit distribution across the entire orchard area. This approach reduces labor costs, enhances the scalability of monitoring, and improves analysis accuracy, making it an effective tool for yield assessment and informed decision-making.

Future research directions include the integration of multispectral data to enhance detection and achieve more accurate classification. Additionally, the introduction of a correction coefficient is proposed, determined by comparing the number of fruits recognized in the images with the actual number of fruits on the tree. The correction coefficient will be calculated experimentally based on field observations, including the quantitative correlation of photographic data with the results of manual fruit counting on a representative sample of trees. This will allow for accounting for fruits hidden from the camera and improving the accuracy of the final yield estimation. To consider all factors, including hidden fruits, the total number of apples is determined by formula 4:

$$N_{\text{total}} = \sum_{i=1}^n \sum_{j=1}^m 1_{ji}^{\text{det}} \cdot C_{ji} \cdot K_{\text{corr},ji} \quad (4)$$

where N_{total} – the total number of fruits across the entire orchard area, pcs., n, m – the number of tiles along the X and Y axes on the orthophotomap, pcs., 1_{ji}^{det} – an indicator of successful object detection in the (i, j) -th tile: $1_{ji}^{\text{det}} = 1$, if $C_{ji} \geq T$, where T – confidence threshold (e.g., 0.5); $1_{ji}^{\text{det}} = 0$, if $C_{ji} < T$, C_{ji} – confidence of the YOLO11 model in correctly detecting an object in the (i, j) -th tile (probability, value from 0 to 1), $K_{\text{corr},ji}$ – the correction coefficient accounting for objects hidden from the camera, unique for each tile, is determined experimentally, considering local conditions such as planting density and the degree of object overlap (e.g., in areas with dense foliage, $K_{\text{corr},ji}$ may range from 1.5 to 2, while in areas with low foliage density, it may be closer to 1).

To address the issue of incomplete fruit coverage in the images caused by the limitations of vertical shooting, it is proposed to supplement the data with images taken at an angle relative to the tree canopy. This approach will allow capturing additional areas of the trees, including partially hidden fruits, and using this data to build 3D models of the trees with the application of dense point cloud algorithms available in Agisoft Metashape. This will provide a more comprehensive representation of the canopy structure, improve object detection, and enhance the accuracy of analysis in dense plantings.

The research was carried out at the expense of a grant from the Russian Science Foundation - Agreement No. 24-76-10071, dated 09.08.2024, <https://rscf.ru/project/24-76-10071/>

References

1. J. W. Jones, *Agricultural Systems* **155** (5), 269-288 (2017)
2. F. J. Pierce, T. V. Elliott, *Computers and Electronics in Agriculture* **61**(1), 32-43 (2008)
3. B. Shurygin, I. G. Smirnov, A. Chilikin, D. O. Khort, A. I. Kutryev, S. Zhukovskaya, A. Solovchenko, *Horticulturae* **8**(12), 1111 (2022)

4. D. O. Khort, A. I. Kutyrev, I. G. Smirnov, N. Andriyanov, R. Filippov, A. Chilikin, M. E. Astashev, E. A. Molkova, R. M. Sarimov, T. A. Matveeva, S. V. Gudkov, *Sustainability* **16**, 10084 (2024)
5. Y. Lin, *Computers and Electronics in Agriculture* **119**, 61-73 (2015)
6. Y. Sun, D. Zhang, X. Guo, H. Yang, *Plants* **12**, 3032 (2023)
7. T. Wu, Z. Miao, W. Huang, W. Han, Z. Guo, T. Li, *Agriculture* **14(11)**, 1958 (2024)
8. S. Kodors, I. Zarembo, G. La cis, L. Litavniece, I. Apeina ns, M. Sondors, A. Pacej, *Drones* **8**, 734 (2024)
9. O. Melnychenko, L. Scislo, O. Savenko, A. Sachenko, P. Radiuk *Sensors*, **24**, 1913 (2024)
10. Y-S Shiu, R-Y Lee, Y-C Chang, *Remote Sensing* **15(3)**, 814 (2023)
11. M. Razaak, H. Kerdegari, V. Argyriou, P. Remagnino *Lecture Notes in Computer Science*, Springer, Cham **11754**, 778-786 (2019)
12. H. Jemaa, W. Bouachir, B. Leblon, A. LaRocque, A. Haddadi, N. Bouguila, *Remote Sensing* **15(14)**, 3558 (2023)
13. S. J. Pan, Q. Yang *IEEE Transactions on Knowledge and Data Engineering* **22(10)**, 1345-1359 (2010)
14. J. Terven, D.-M. Córdova-Esparza, J.-A. Romero-González *Machine Learning and Knowledge Extraction* **5**, 1680-1716 (2023)